Crew 1.0 - Configuration Reference

This document has two parts. The first is short and hand-written: how crew's two config files combine, how to see what is in force, and the common setups. The second is generated from the code and lists every key.

How the Two Files Combine

crew reads two JSON files:

- `~/.claude/crew/config.json` - the **machine-global** file. Your personal defaults for every repo on this machine: providers and models, notifications, auto-clear, the guards, how far autopilot may go.
- `.crew/config.json` - the **repo** file. Facts about one checkout: the tracker, the board, scope enforcement, production declarations.

The repo file wins over the machine file, which wins over crew's defaults. Four exceptions, each named in the Layer column below:

- A **repo-only** key in the machine file takes effect nowhere. It is pruned on read and refused on write.
- A **ratcheted** key (`install.policy`, each guard `guards.profile` and `guards.overrides` expand to, `change.requireForProduction`, `environments.prodUnattended`) resolves to the narrower of the two values, so a cloned repo cannot widen what your machine allows.
- A **machine-armed** key (`resume.auto`, `context.autoClear.enabled`) can be turned on only in the machine file. A repo can only switch it off.
- A **personal** key (the five `autopilot.*` keys) resolves per key to the stricter of the layers that set it; a layer that says nothing imposes nothing. A machine value is your default for every repo, which a repo may narrow and never widen.

A `null` in the repo file over a machine value inherits the machine value. The `/crew:init` template writes every key except the personal ones, so it does not shadow your machine defaults for those; a repo `/crew:init` set up before that still spells `autopilot.mode: "off"`, which holds a machine `plan` down until you remove it (`--explain --all` names it as a `shadow:`).

Seeing What Is in Force

- `/crew:config --show` prints every machine-settable key, its value and the layer it came from, and names any key that takes effect nowhere.
- `/crew:config` with no argument is a menu that writes either file through the validated path. It shows a dry run first, refuses unknown keys and out-of-range values, and marks a widening with `!`.
- From a shell: `python3 plugin/crew/hooks/scripts/crew_config.py --explain` is the same table, `--explain --all` adds every repo-only key and names each shadowing repo value, and `--models` is the per-role provider table.

Backups and Rebuilding a Lost Config

Every write a crew script makes to either file first copies the old bytes to `~/.claude/crew/backups/` (the newest 20 per file kept) and is refused when that copy fails; a hand edit is not backed up. `crew_config.py --backups` lists the machine file's (add `--repo` for the repo file's) and `--restore <stamp> --apply` puts one back. Your non-default values are kept in `~/.claude/crew/profile.json` (and in your vault when `memory.vaultPath` is set), so `--rebuild --repo` or `--rebuild --global` regenerates a lost or corrupt file from the template plus that profile, as a dry run until `--apply`.

Common Setups

- **Personal defaults once, for every repo.** Put providers, models, `notify.*`, the guards and the `autopilot.*` keys in `~/.claude/crew/config.json`. Leave repo files to repo facts.
- **Self-approval under autopilot.** Set `autopilot.mode: "plan"` and choose `autopilot.approval` (`human`, `self` or `risk`) in either file, and set `scope.allowCliApproval: true` in the repo file, which is the only file that key is read from. Review acceptance and production deploys still stop for you.
- **Notifications.** In the machine file, set `notify.provider` and the name of the environment variable holding the token in `notify.tokenEnv`. The secret stays in your environment, never in the file. `tokenEnv` is machine-only: a repo file's value is ignored with a notice.

The key list is `plugin/crew/CONFIG.md`, which ships with the plugin; the reasoning behind the keys is in `docs/crew/config-rationale.md`.

The Human-Only Opt-Out (`humanOnly`)

`humanOnly.approve` and `humanOnly.promoteMarker` are the owner's opt-out of two human-only rules: `/crew:approve`, and the production promote marker `.crew/.approved-<env>-<sha>`. `null` and `true` mean human-only. A `false` takes effect only when all of these hold (`crew_humanonly.resolve`):

- it is in the machine file, or in a repo `.crew/config.json` that git does not track (a tracked or committed repo file can never opt out; "could not tell" counts as tracked);
- neither layer says `true` (a `true` anywhere vetoes, the stricter wins);
- it was written by a double confirmation: `crew_confirm_config.py propose` records the change and prints an id with two lines the agent relays to you word for word (the change, and the exact command; crew 1.3.70), and the owner types `/crew:confirm-config <change_id>` twice, in two prompts, within 15 minutes. `crew_config.py --set` and `/crew:config` refuse a `false` (and a repo `null` or removal, which drops a veto); a `false` written by hand reads as human-only.

What is live (crew 1.3.33). The prompt hook (`approval_hook.py`) now records `/crew:confirm-config <change_id>` from a prompt you typed, so a double confirmation can complete, and `humanOnly.approve: false` takes effect through ONE decision point, `crew_ticket.cli_approval_allowed`: with an opt-out in force a `cli`-recorded approval receipt counts as if `scope.allowCliApproval` were `true`, for the scope guard, the Stop audit and every autopilot reader at once. It does NOT let the session run `crew_ticket.py approve` (the scope guard still refuses that text); the session's route is `crew_autopilot.py approve`, which still needs `autopilot.approval self` or `risk` and `autopilot.mode plan` or `backlog`. Anything unclear - the module missing, the resolver raising or answering anything but exactly `humanOnly: false` - reads as human-only.

Finding the change id (crew 1.3.69). Type `/crew:confirm-config` with no id (or with an id that is wrong, expired or already used) and the hook refuses it, records nothing, and lists the proposals in this machine's store (`~/.claude/crew/confirm-config/`): each pending one's id, key, current value -> `false`, layer and file, age, time left and confirmations so far, then up to five from the last day that can no longer be confirmed. With none pending it says so, and why one may be missing: none was made, it expired, it was already applied, cancelled or refused, or it was proposed under another home directory (WSL rather than Windows has its own store). The store is per machine and home directory, not per repository, so a proposal made in one repo or worktree is listed from any other. The listing never confirms: even with exactly one pending proposal a bare `/crew:confirm-config` records nothing, because "the only pending one" is a choice whoever ran `propose` makes, and the id you type is what binds the change. A store crew cannot read is reported as "could not tell", never as none pending.

Still human-only (crew 1.3.33). No hook reads `humanOnly.promoteMarker`: the promote marker is human-only by instruction alone, and no hook checks who created it. `crew_confirm_config.py resolve` shows what each key would resolve to.

Who can write the receipts. The double confirmation is recorded only by the prompt hook, from your own prompt. The scope guard (when `scope.mode` is not `off`) refuses every Write/Edit under `~/.claude/crew/confirm-config/` and every shell command that names that store beside a writing word. That is a textual tripwire, not a sandbox: a session that sets out to forge a receipt (`python -c`, an encoded command, a variable holding the path) still can, and `scope.mode: off` has no tripwire at all. Treat `humanOnly` opt-outs as meaningful only with `scope.mode block`.

`/crew:approve-mass <id>` (crew 1.3.36). Your yes to ONE blocked mass operation, typed by you and recorded by the same prompt hook in `<git-common-dir>/crew/mass-approvals/<id>.json`, good for 15 minutes. The directory is under `<git-common-dir>/crew/`, which the scope guard refuses to every Write/Edit and every shell command naming it, so the session cannot write the file. The cloud guard (`cloud_guard.py`) prints the `<id>` with its stop and reads the file; see "Mass operations".

Mass Operations (`guards.mass`, Crew 1.3.36)

The rule for an autopilot that runs without a human: full automation, except a mass delete, a mass create or a mass change, which wait for your yes. The stop is armed by `guards.mass.enabled: true` (either layer) or by a double-confirmed `humanOnly` opt-out, and by nothing else; it is off by default. It rides on the cloud guard's hook, judges every shell command, and works even where `guards.cloudGuard` is `off`.

Limit	Default	Counts
`deleteCount`	10	files a command removes
`createCount`	20	files it adds
`changeCount`	30	files it rewrites

A count AT the limit stops. The defaults were measured on this repo's own history (104 days, 623 merged PRs: a PR adds a median 0 and p95 16 files, changes a median 5 and p95 30, deletes none in all but 15). `guards.mass.overrides.<files|git|terraform>.<limit>` replaces one limit for one kind of command; with nothing set `terraform` is delete 3, create 10, change 10 (inferred - the history held no terraform). The machine file sets the limits

and `guards.mass.exempt`; the repo file can lower a limit and turn the stop on (`enabled` is on if either layer says so), so a cloned repo can tighten the stop and never loosen yours. `guards.mass.exempt` (machine file only) lists repo-relative globs of generated files that never count; node_modules, `__pycache__`, `.venv`, caches, `build`, `dist`, `.work` and worktrees (including `~/.claude/crew/worktrees/<repo>/<name>`) are always exempt.

What it sees: `rm`, `Remove-Item`, `git clean -f`, `git rm`, `find -delete`, `rsync --delete` (delete); `cp`, `mv`, `rsync`, `Copy-Item`, `touch` (create and change); `sed -i` and `git reset --hard` / `checkout -- <paths>` / `restore` (change); `git commit`, `git push` and `gh pr create`, which count the added, deleted and changed files of the commit or of the whole branch against its base - so Write and Edit work is counted where it is about to be shared; `terraform apply <plan>` (the plan's own changes), `destroy` and `apply -auto-approve`; and an unfiltered SQL `DELETE` or `UPDATE`.

A count that cannot be taken - a variable in a path, a path after a `cd`, `find -delete`, `xargs rm`, a failed `git`, an unfiltered SQL statement (no command text says how many rows it hits) - is "could not tell", and could not tell stops. It does NOT see a script (`bash cleanup.sh`, `python -c`, `make clean`), an MCP server, the Write and Edit tools before they are committed, `tar -x` or `unzip`, `git merge` or `pull`, or many small commands each under a limit: a tripwire in front of native permissions, not a sandbox.

A stopped command prints an id. Attended, the stop is an `ask`; unattended it is a `deny`. Type `/crew:approve-mass <id>` in the session, then re-run the exact command. One id covers every limit that command trips.

Reference (Generated)

Generated from the code by `python3 docs/guides/crew/src/config_reference.py --write`. Do not edit by hand: `python3 scripts/check-marketplace.py` fails when this file is stale.

106 keys: 68 settable in the machine-global file, 38 repo-only.

Columns:

- **Layer**: which file may set the key.
- **Default**: what `default_config()` returns. Where the machine default differs, both are shown.
- **Values**: the values crew accepts, read from the same tuple the validator reads. "not validated" means crew checks nothing and names the file that reads the key.
- **Since**: the first crew version whose template declared the key. Keys in the first template (crew 0.11.0) read "0.11.0 or earlier".

Layers

- **repo**: only `.crew/config.json` may set it; the global file's value is pruned on read and refused on write.
- **both**: either file may set it; the repo value wins over the machine one.
- **both, ratchet**: either file may set it, and the NARROWER of the two wins (`crew_guards.RATCHETED_KEYS`). A repo can tighten what the machine allows, never loosen it.
- **both, widening warned**: either file may set it and the repo wins, but a write that widens it is flagged (`crew_config._RATCHETED`).
- **both, stricter wins**: a personal key (`crew_guards.PERSONAL_KEYS`): either file may set it, and where both do the STRICTER value wins; a layer that is silent (absent or `null`) imposes nothing, so a machine value is your default for every repo and a repo may only narrow it.
- **machine-arms**: only the global file can turn it on (exactly `true`); a repo may only veto it with `false` (`crew_config.REPO_VETO_ONLY`).
- **machine-only**: read from the global file alone; a repo's own value is never consulted.
- **human-only opt-out**: a `humanOnly` key (`crew_humanonly`): `false` opts out only from the machine file or an untracked repo file, only after the owner types `/crew:confirm-config <change_id>` twice, and a `true` in either layer vetoes it. Not acted on by any hook yet (see the section below).

Top-Level Keys

Setting	Layer	Default	Values	Since	Summary
`schema`	repo	`8`	not validated - read by `plugin/crew/hooks/scripts/crew_state.py` (expects integer)	0.11.0 or earlier	Config schema version; `/crew:migrate` upgrades an older one.
`tracker`	repo	`"files"`	`files` | `obsidian` | `jira` | `sdp`	0.11.0 or earlier	Where tickets live; `crew_tracker.resolve` answers `could not tell` when two configs disagree.

Setting	Layer	Default	Values	Since	Summary
`verifyGate`	repo	`true`	not validated - read by `plugin/crew/hooks/scripts/verify-gate.sh` (expects boolean)	0.11.0 or earlier	Run the Stop verify gate.

Qa

Setting	Layer	Default	Values	Since	Summary
`qa.provider`	both	`"auto"`	`auto` | `claude` | `codex` | `copilot` | `kimi`	0.11.0 or earlier	Who reviews: `auto` walks `qa.order`; a named provider is used as-is.
`qa.order`	both	`["codex", "kimi", "copilot", "claude"]`	list of: `claude` | `codex` | `copilot` | `kimi`	0.14.6	The reviewers `auto` tries, in order. A list is one leaf, replaced wholesale.
`qa.fallback`	both	`"claude-sonnet-5"`	not validated - read by `plugin/crew/commands/review.md` (expects model id)	0.16.6	Model a gone pin falls back to, on its family's provider (L-0712).
`qa.codex.model`	both	`null`	not validated - read by `plugin/crew/commands/review.md` (expects string or null)	0.14.6	Codex model for review; null passes no flag.
`qa.codex.reasoningEffort`	both	`null`	`none` | `minimal`	0.14.	Codex

Setting	Layer	Default	Values	Since	Summary
			| `low` | `medium` | `high` | `xhigh` | `max` (listed in `plugin/crew/commands/review.md`; not validated)	6	reasoning effort for review; null passes no flag.
`qa.copilot.model`	both	`null`	not validated - read by `plugin/crew/commands/review.md` (expects string or null)	0.14.6	Copilot model for review; null uses the CLI default.
`qa.kimi.model`	both	`null`	not validated - read by `plugin/crew/commands/review.md` (expects string or null)	1.0.39	Kimi Code model id for review (`k3`, `kimi-for-coding`, ...); null uses the CLI's own default.
`qa.roles`	both	`{}`	object of role pins; each pin's provider is checked (checked in `plugin/crew/hooks/scripts/crew_config.py`)	0.16.6	Per-role reviewer pins, `qa.roles.<role>.provider` / `.model`; a pin beats the provider block for that role.

Dev

Setting	Layer	Default	Values	Since	Summary
`dev.provider`	both	`"claude"`	`claude` | `codex` | `copilot` | `kimi`	0.14.6	Who implements.

Setting	Layer	Default	Values	Since	Summary
`dev.fallback`	both	`"claude-sonnet-5"`	not validated - read by `plugin/crew/skills/crew-providers/SKILL.md` (expects model id)	0.16.6	Model a gone dev pin falls back to, on its family's provider.
`dev.codex.model`	both	`null`	not validated - read by `plugin/crew/skills/crew-providers/SKILL.md` (expects string or null)	0.14.6	Codex model f implementatio
`dev.codex.reasoningEffort`	both	`null`	`none` | `minimal` | `low` | `medium` | `high` | `xhigh` | `max` (listed in `plugin/crew/commands/review.md`; not validated)	0.14.6	Codex reasoning effo for implementatio
`dev.copilot.model`	both	`null`	not validated - read by `plugin/crew/skills/crew-providers/SKILL.md` (expects string or null)	0.14.6	Copilot model for implementatio
`dev.kimi.model`	both	`null`	not validated - read by `plugin/crew/skills/crew-providers/SKILL.md` (expects string or null)	1.0.39	Kimi Code model id for implementatio as `qa.kimi.model`.
`dev.roles`	both	`{}`	object of role	0.16.	Per-role dev

Setting	Layer	Default	Values	Since	Summary
			pins; each pin's provider is checked (checked in `plugin/crew/hooks/scripts/crew_config.py`)	6	pins, as `qa.roles`.

secondOpinion

Setting	Layer	Default	Values	Since	Summary
`secondOpinion.provider`	both	`"none"`	not validated - read by `plugin/crew/commands/plan.md` (expects string)	0.11.0 or earlier	Second-opinion provider for plans; `none` is off.
`secondOpinion.mode`	both	`"cli"`	not validated - read by `plugin/crew/skills/crew-providers/SKILL.md` (expects string)	0.11.0 or earlier	How the second opinion is reached.
`secondOpinion.model`	both	`null`	not validated - read by `plugin/crew/skills/crew-providers/SKILL.md` (expects string or null)	0.11.0 or earlier	Second-opinion model.

Jira

Setting	Layer	Default	Values	Since	Summary
`jira.project`	repo	`null`	not validated -	0.11.0	Jira project key. No

Setting	Layer	Default	Values	Since	Summary
			read by `plugin/crew/commands/jira-sync.md` (expects string or null)	or earlier	consumer found (docs/crew/config-rationale.md).
`jira.cloudId`	repo	`null`	not validated - read by `plugin/crew/commands/jira-sync.md` (expects string or null)	0.19.10	Jira cloud id, cached by `/crew:jira-sync`; read by nothing.

Obsidian

Setting	Layer	Default	Values	Since	Summary
`obsidian.vaultPath`	repo	`null`	not validated - read by `plugin/crew/hooks/scripts/crew_tracker.py` (expects path or null)	0.11.0 or earlier	Vault holding the board; falls back to `memory.vaultPath` and must hold `.obsidian/`.
`obsidian.boardDir`	repo	`null`	not validated - read by `plugin/crew/hooks/scripts/crew_tracker.py` (expects path or null)	0.11.0 or earlier	Board folder inside the vault (relative, no `..`).

Memory

Setting	Layer	Default	Values	Since	Summary
`memory.mode`	both	`"repo"`	not validated - read by `plugin/crew/skills/crew-memory/SKILL.md` (expects string)	0.11.0 or earlier	Where memory lives (`repo`, or a vault).
`memory.vaultPath`	both	`null`	not validated - read by `plugin/crew/hooks/scripts/crew_recall.py` (expects path or null)	0.11.0 or earlier	The Obsidian vault used for memory.

Context

Setting	Layer	Default	Values	Since	Su
`context.enabled`	repo	`true`	not validated - read by `plugin/crew/hooks/scripts/context-watch.sh` (expects boolean)	0.11.0 or earlier	Ru co wa
`context.warnAt`	repo	`0.4`	not validated - read by `plugin/crew/hooks/scripts/context-watch.sh` (expects number)	0.11.0 or earlier	Fr th at th wa wr ar cl de si (w ar va

Setting	Layer	Default	Values	Since	Su
					ke
`context.reserveTokens`	repo	`0`	not validated - read by `plugin/crew/hooks/scripts/context-watch.sh` (expects integer or null)	0.11.0 or earlier	To ba th nu of
`context.autoClear.enabled`	machine-arms	`null`	`null` | `true` | `false` (checked in `plugin/crew/hooks/scripts/crew_autocycle.py`)	0.19.10	Au af wr On m ca (e tr re ve
`context.autoClear.method`	both	`"auto"`	`auto` | `none` | `notify` | `tmux` | `xdotool` | `wtype`; per OS: linux: auto, none, notify, tmux, xdotool, wtype; macos: auto, none, notify, tmux; windows: auto, none, notify; windows-bash: auto, none, notify, tmux	0.19.10	Ho is au pe pl m OS st au do
`context.autoClear.delaySeconds`	both	`3`	number (coerced in `plugin/crew/hook`	0.19.10	De be ty

Setting	Layer	Default	Values	Since	Su
			s/scripts/crew_autocycle.py)		ur va ar as
context.autoClear.minHandoffLines	both	5	number (coerced in plugin/crew/hooks/scripts/crew_autocycle.py)	0.19.10	Sk ha m cl af no re
context.autoClear.onlyRepos	machine-only	null	list of absolute repo paths, or null (coerced in plugin/crew/hooks/scripts/crew_autocycle.py)	1.0.25	Na or m or na no lis or re a ar no
context.autoClear.onlySessions	machine-only	null	list of session ids, or null (coerced in plugin/crew/hooks/scripts/crew_autocycle.py)	1.0.25	As on fo id bo bo m
context.autoClear.wrapUp	machine-arms	null	null | true | false (checked in plugin/crew/hooks/scripts/crew_autocycle.py)	1.0.334	Au up au (T th be

Setting	Layer	Default	Values	Since	Su
					th pr ar cl fo re O m ar (e tr wi en ar re ve

Resume

Setting	Layer	Default	Values	Since	Summary
`resume.auto`	machine-arms	`null`	`null` | `true` | `false` (checked in `plugin/crew/hooks/scripts/crew_resume.py`)	1.0.40	Auto-resume after `/clear` or a manual `/compact`. Only the machine file can arm it (exactly `true`); a repo `false` vetoes it.

humanOnly

Setting	Layer	Default	Values	Since	Summary
`humanOnly.approve`	human-only opt-out	`null`	`null` | `true` | `false` (checked in `plugin/crew/hooks/scripts/crew_h`	1.3.9	The owner's opt- `/crew:approve` be null or `true` keep `false` opts out or

Setting	Layer	Default	Values	Since	Summary
			umanonly.py)		machine file or a file, only after /c config twice, and layer vetoes. cre opt-out in force a approval counts (crew_ticket.cli).
humanOnly.promoteMarker	human-only opt-out	null	null | true | false (checked in plugin/crew/hook s/scripts/crew_h umanonly.py)	1.3.9	The same opt-ou production prom .crew/.approved- under the same no hook acts on hook today chec the marker.

Worktrees

Setting	Layer	Default	Values	Since	Summary
worktrees.root	both	null	absolute path or null (coerced in plugin/crew/hooks /scripts/crew_wor ktrees.py)	1.3.19	The base directory for this repo's git worktrees, OUTSIDE the repo: they go in <root>/<repo>-<digest>/<name>. null is ~/.claude/crew/workt rees; ~ is expanded, a relative path or one inside the repo is refused (never replaced by the default). It places the worktrees crew places (/crew:emergency, crew_worktrees.py

Setting	Layer	Default	Values	Since	Summary
					`path`) and, once the owner registers the opt-in `WorktreeCreate` hook, the ones Claude Code creates; without the hook Claude Code keeps using `.claude/worktrees/`.

Promote

Setting	Layer	Default	Values	Since	Summary
`promote.deadlineSeconds`	both	`null`	number of seconds, or null (coerced in `plugin/crew/hooks/scripts/_promote_gate.py`)	1.3.41	How many seconds the promote gate (`promote-gate.sh`) may run, counted from the moment its shell began, before it gives up and blocks (a hook that times out is not a block, so the gate stops itself first). null derives it from the guard's hook timeout less a 4 s margin: 16 s of the 20 s row. A number above the timeout less

Setting	Layer	Default	Values	Since	Summary
					2 s (18 s of 20) is clamped to that, so the setting can never let the gate outlive its hook; zero, a negative, a string or a boolean reads as null. The hook timeout is the guard's row in the dispatcher's table, not a setting.

Reaper

Setting	Layer	Default	Values	Since	Summary
`reaper.enabled`	both	`false`	boolean (coerced in `plugin/crew/hooks/scripts/crew_reaper.py`)	1.3.50	Whether scheduled and session-start passes of the orphan reaper (`crew_reaper.py --scheduled`) do anything. false by default; a manual `crew_reaper.py` run always works and is a dry run unless given `--apply`. Read from the

Setting	Layer	Default	Values	Since	Summary
					machine file only: a repo value is ignored, because the reaper ends processes on this machine.
`reaper.dryRun`	both	`true`	boolean (coerced in `plugin/crew/hooks/scripts/crew_reaper.py`)	1.3.50	When true (the default) a scheduled or session-start pass only logs what it would end to `~/.claude/crew/reaper.log`; set false to let it end processes. Only a scheduled pass reads it; a manual run ends processes only with `--apply`.
`reaper.spinCpuPct`	both	`50`	percent of one core (coerced in `plugin/crew/hooks/scripts/crew_reaper.py`)	1.3.50	An orphan is SPINNING when it used at least this percentage of one core over the reaper's short sample (5 s) and is older than `spinMinAgeMin`.

Setting	Layer	Default	Values	Since	Summary
					A number above 0; anything else reads as 50.
`reaper.spinMinAgeMin`	both	`15`	number of minutes (coerced in `plugin/crew/hooks/scripts/crew_reaper.py`)	1.3.50	Minutes an orphan must be running before the spinning rule (and the self-matching `until pgrep -f` watcher rule) may end it. A number above 0; anything else reads as 15.
`reaper.blockedMinAgeHours`	both	`2`	number of hours (coerced in `plugin/crew/hooks/scripts/crew_reaper.py`)	1.3.50	Hours an orphan using under 1% of a core must be running before it counts as BLOCKED and may be ended. A number above 0; anything else reads as 2.
`reaper.protectedRegex`	both	`null`	regular expression, or null (coerced in `plugin/crew/hooks/scripts/crew_reaper.p`	1.3.50	A regular expression; a process whose image name or command line matches is

Setting	Layer	Default	Values	Since	Summary
			`y`)		never ended, it REPLACES the built-in list (claude, node, MCP servers, ai-connect, psmux/tmux, tailscale, language servers, obsidian, perplexity, vscode, wsl/docker/vm, Defender, ssh). null keeps the built-in list; an invalid expression reads as null.
`reaper.intervalMin`	both	`10`	whole number of minutes (coerced in `plugin/crew/hooks/scripts/crew_reaper.py`)	1.3.50	Minutes between runs of the scheduled task `crew_reaper.py --install-task` creates, and the least gap between two session-start passes. A whole number from 1; anything else reads as 10. The task is never installed by default.

Setting	Layer	Default	Values	Since	Summary
`reaper.onSessionStart`	both	`false`	boolean (coerced in `plugin/crew/hooks/scripts/crew_reaper.py`)	1.3.50	When true and `reaper.enabled` is true, the SessionStart context hook starts one detached reaper pass (at most one per `intervalMin`) and does not wait for it. false by default.

Emergency

Setting	Layer	Default	Values	Since	Summary
`emergency.standDown`	repo	`true`	not validated - read by `plugin/crew/hooks/scripts/_common.sh` (expects boolean)	0.11.0 or earlier	Whether a declared incident may stand gates down; `false` forbids it.
`emergency.maxTtlMinutes`	repo	`480`	integer (coerced in `plugin/crew/hooks/scripts/crew_incident.py`)	0.11.0 or earlier	Longest incident lifetime; never below `ttlMinutes`.

Notify

Setting	Layer	Default	Values	Since	Summary
`notify.provider`	both	`null`	telegram, teams, none or null; any	0.11.0 or earlie	Where notifications go: `telegram`,

Setting	Layer	Default	Values	Since	Summary
			other value sends nothing (coerced in `plugin/crew/hooks/scripts/crew_notify.py`)	r	`teams` or `none` (off). A repo null inherits the machine provider; a repo `none` opts this repo out.
`notify.tokenEnv`	machine-only	`null`	not validated - read by `plugin/crew/hooks/scripts/crew_notify.py` (expects string or null)	0.11.0 or earlier	Name of the environment variable holding the Telegram bot token; null may be filled from the notify skill's `bot_token_env`. Honoured from the machine file only.
`notify.chatId`	both	`null`	not validated - read by `plugin/crew/hooks/scripts/crew_notify.py` (expects string or null)	0.11.0 or earlier	Telegram chat id; null may be filled from the notify skill's `chat_id`, and the skill's example value counts as unset.
`notify.events`	both	`["blocker", "deploy", "question"]`	list of event names; an unknown name is dropped with a notice (coerced in `plugin/crew/ho`	0.11.0 or earlier	Events that notify: `deploy`, `question`, and `blocker` (T-0060's four blocker reasons). The

Setting	Layer	Default	Values	Since	Summary
			oks/scripts/crew_notify.py)		pre-1.0 names gate, waiting, phase, review and done are mapped with a notice. A list is one leaf.

Platform

Setting	Layer	Default	Values	Since	Summary
platform.os	repo	null	not validated - read by plugin/crew/hooks/scripts/crew_platform.py (expects string or null)	0.11.0 or earlier	Detected OS, stamped by platform-sync.
platform.wsl	repo	null	not validated - read by plugin/crew/hooks/scripts/crew_platform.py (expects boolean or null)	0.11.0 or earlier	Detected WSL, stamped by platform-sync.
platform.shell	repo	null	not validated - read by plugin/crew/hooks/scripts/crew_platform.py (expects string or null)	0.11.0 or earlier	Detected shell, stamped by platform-sync.
platform.windowsHostIp	repo	null	not validated - read by plugin/crew/hooks/scripts/crew_platform.py (expects string or null)	0.11.0 or earlier	Windows host IP seen from WSL.

Pm

Setting	Layer	Default	Values	Since	Summary
`pm.authority`	both, widening warned	`"report-only"`	`report-only` | `act` | `autonomous`	0.11.0 or earlier	How much the PM may do unasked. A widening is warned about; the two layers combine by precedence.
`pm.maxDispatches`	both	`3`	integer (coerced in `plugin/crew/hooks/scripts/crew_state.py`)	0.11.0 or earlier	Roles the PM may dispatch in one pass.

Graph

Setting	Layer	Default	Values	Since	Summary
`graph.out`	repo	`"graphify-out"`	not validated - read by `plugin/crew/hooks/scripts/crew_state.py` (expects path)	0.11.0 or earlier	Where the code graph is written.

Install

Setting	Layer	Default	Values	Since	Summary
`install.policy`	both, ratchet	`"manual"`	`manual` | `ask` | `auto` (ratchet: narrower layer wins; listed narrowest first)	0.19.18	Whether crew may install a missing prerequisite.

Guards

Setting	Layer	Default	Values	Since	Summary
`guards.profile`	both, ratchet	`"off"`	`off` | `standard` | `strict`	1.2.62	One word for the twelve guards: `off` (today's defaults), `standard` or `strict`. Expands per layer; each guard's ratchet then runs on the expanded values.
`guards.overrides`	both, ratchet	`{}`	object of guard name to tier (checked in `plugin/crew/hooks/scripts/crew_guards.py`)	1.2.62	`guards.overrides.<guard>`: one guard's tier, replacing the profile's value for it. Each entry is checked against that guard's own tiers; a bad one reads as the floor.
`guards.mass.enabled`	both	`false`	boolean (coerced in `plugin/crew/hooks/scripts/crew_mass.py`)	1.3.36	Turns the mass delete / create / change stop on (it is also armed by a double-confirmed `humanOnly` opt-out). On if EITHER layer says exactly `true`; off by default.
`guards.mass.deleteCount`	both	`10`	integer (1 or more) (coerced in `plugin/crew/hooks/scripts/crew_mass.py`)	1.3.36	Files a command may delete before it waits for `/crew:approve-mass`; a count at the limit stops. The machine file sets it; the repo file can only lower it; a value that is not a positive integer reads as the default.
`guards.mass.createCount`	both	`20`	integer (1 or more)	1.3.3	Files a command may create before it waits

Setting	Layer	Default	Values	Since	Summary
			(coerced in `plugin/crew/hooks/scripts/crew_mass.py`)	6	for `/crew:approve-mass`; a count at the limit stops. The machine file sets it; the repo file can only lower it; a value that is not a positive integer reads as the default.
`guards.mass.changeCount`	both	`30`	integer (1 or more) (coerced in `plugin/crew/hooks/scripts/crew_mass.py`)	1.3.36	Files a command may rewrite before it waits for `/crew:approve-mass`; a count at the limit stops. The machine file sets it; the repo file can only lower it; a value that is not a positive integer reads as the default.
`guards.mass.overrides`	both	`{}`	object of category to {deleteCount, createCount, changeCount} (checked in `plugin/crew/hooks/scripts/crew_mass.py`)	1.3.36	`guards.mass.overrides.<files|git|terraform>.<deleteCount|createCount|changeCount>`: one limit for one kind of command. Unset, `terraform` is delete 3, create 10, change 10 (inferred, not measured); the others use the limits above.
`guards.mass.exempt`	both	`[]`	list of globs (coerced in `plugin/crew/hooks/scripts/crew_mass.py`)	1.3.36	Repo-relative globs of generated files that never count toward a limit, read from the machine file only (a glob can only loosen the stop), on top of the

Setting	Layer	Default	Values	Since	Summary
					built-in regenerable directories (node_modules, **pycache**, .venv, caches, build, dist, .work, worktrees).

Production

Setting	Layer	Default	Values	Since	Summary
`production.databases`	repo	`[]`	not validated - read by `plugin/crew/hooks/scripts/crew_config.py` (expects list of globs)	0.19.30	Globs naming production databases.
`production.hosts`	repo	`[]`	not validated - read by `plugin/crew/hooks/scripts/crew_config.py` (expects list of globs)	0.19.30	Globs naming production hosts.

Cloud

Setting	Layer	Default	Values	Since	Summary
`cloud.awsProfiles`	repo	`[]`	not validated - read by `plugin/crew/hooks/scripts/cloud_guard.py` (expects list of names)	1.0.25	AWS profiles this repo may use.
`cloud.awsRegions`	repo	`[]`	not validated - read by	1.0.2	AWS regions this repo may

Setting	Layer	Default	Values	Since	Summary
			`plugin/crew/hooks/scripts/cloud_guard.py` (expects list of names)	5	use.
`cloud.azureSubscriptions`	repo	`[]`	not validated - read by `plugin/crew/hooks/scripts/cloud_guard.py` (expects list of names)	1.0.25	Azure subscriptions this repo may use.

Environments

Setting	Layer	Default	Values	Since	Summary
`environments.nonProd`	repo	`[]`	not validated - read by `plugin/crew/hooks/scripts/crew_config.py` (expects list of globs)	1.0.37	Terraform targets that are not production and may run unattended.
`environments.prodUnattended`	both, ratchet	`false`	`false` | `true` (ratchet: narrower layer wins; listed narrowest first)	1.0.37	Whether production terraform may run unattended; `true` only when both layers say so.
`environments.workflows`	repo	`{}`	not validated - read by `plugin/crew/hooks/scripts/crew_config.py`	1.1.16	Deploy workflow globs, each mapped to its environment

Setting	Layer	Default	Values	Since	Summary
			(expects object of glob to string)		or `input:<name>`.
`environments.preProduction`	repo	`{}`	not validated - read by `plugin/crew/hooks/scripts/crew_preprod.py` (expects object of exact name to {reason, targets})	1.3.20	Environments labelled production that are not live yet: exact name -> `reason` (+ optional exact `targets`); judged non-production by the cloud guard, with an audit line.

Change

Setting	Layer	Default	Values	Since	Summary
`change.requireForProduction`	both, ratchet	`false`	`true` | `false` (ratchet: narrower layer wins; listed narrowest first)	0.19.31	Require an approved change request before `/crew:promote production`: a repo may turn it on, never off.

Git

Setting	Layer	Default	Values	Since	Summary
`git.forbiddenTrailers`	both	`[]`	list of trailer tokens (letters, digits and `-`, no `:`) (checked in `plugin/crew/hooks/scripts/crew_trailers.py`)	1.0.328	Commit trailer tokens the owner forbids, reported by `/crew:done`. The two layers combine by union, so a repo can add a token and never remove the machine owner's; a value that is not a list of tokens makes the list unknown, never empty (docs/crew/config-rationale.md).

Scope

Setting	Layer	Default	Values	Since	Summary
`scope.mode`	repo	`"off"`	`off` | `report` | `block` | `auto`	1.0.25	Whether the scope guard enforces a ticket's Touch list; a value outside these fails closed to `block`.
`scope.allowCliApproval`	repo	`false`	`false` | `true` (checked in `plugin/crew/hooks/scripts/crew_ticket.py`)	1.0.25	Whether a CLI-written approval receipt counts; only exactly `true` allows it. `autopilot.approval: self` or `risk` needs it, and `autopilot.mode: plan` or `backlog`, to approve anything. A hand edit by the owner in the repo's

Setting	Layer	Default	Values	Since	Summary
					`.crew/config.json` (`"scope": {"allowCliApproval": true}`); `crew_config.py --set` refuses it.

Autopilot

Setting	Layer	Default	Values	Since	Summ
`autopilot.mode`	both, stricter wins	`"off"`	`off` | `plan` | `backlog` (checked in `plugin/crew/hooks/scripts/crew_autopilot.py`); personal: listed strictest first, the stricter wins	1.0.41	Only t `plan` (and `ba` goal's deper inside `/crew:` anyth off, wi
`autopilot.maxPhases`	both, stricter wins	`12`	positive integer (checked in `plugin/crew/hooks/scripts/crew_autopilot.py`); personal: the smaller wins	1.0.41	Phase take; positiv as 12,
`autopilot.deploy`	both, stricter wins	`"none"`	`none` | `nonprod` | `all`; personal: listed strictest first, the stricter wins	1.0.42	Where run wi anyth `none`.
`autopilot.approval`	both, stricter wins	`"risk"`	`human` | `risk` | `self`; personal: listed strictest first, the stricter wins	1.0.42	Who a under anyth `human`. also n `autopi` `backlo`

Setting	Layer	Default	Values	Since	Summ
					`scope`. `true` (a repo's `.crew/` refuse each u its val file th under risk th
`autopilot.questions`	both, stricter wins	`"risk"`	`human` | `risk` | `self`; personal: listed strictest first, the stricter wins	1.0.42	Who a open autop reads
`autopilot.maxAutoReplans`	repo	`0`	non-negative integer (checked in `plugin/crew/hooks/scripts/crew_autopilot.py`)	1.0.339	Succe autop reject round itself; anyth negat as 0, a with a
`autopilot.ship`	repo	`"merge"`	`pr` | `merge`	1.0.349	After pushe PR; `me` (a me the re allow. reads warni
`autopilot.sleep.schedule`	repo	`null`	HH:MM-HH:MM or null (checked in `plugin/crew/hooks/scripts/crew_sleep.py`)	1.0.332	A nigh `HH:MM-` local t midni two sl

Setting	Layer	Default	Values	Since	Summ
					apply. could stricte applie
`autopilot.sleep.approval`	repo	`null`	`null` | `human` | `self` | `risk` (checked in `plugin/crew/hooks/scripts/crew_sleep.py`)	1.0.332	`autopi` inside windo day va else c the st warni
`autopilot.sleep.questions`	repo	`null`	`null` | `human` | `self` | `risk` (checked in `plugin/crew/hooks/scripts/crew_sleep.py`)	1.0.332	`autopi` inside windo day va else c the st warni
`autopilot.sleep.deploy`	repo	`null`	`null` | `nonprod` | `none` (checked in `plugin/crew/hooks/scripts/crew_sleep.py`)	1.1.21	`autopi` the sl (L-065 day va `none` r anyth includ with a an eff as `non` never asleep
`autopilot.maxTicketsPerRun`	repo	`3`	positive integer (checked in `plugin/crew/hooks/scripts/crew_autopilot_backlog.py`)	1.1.21	Ticket (one s the ne run. A positiv

Setting	Layer	Default	Values	Since	Summ
					as 3, w
`autopilot.maxTokensPerSession`	repo	`2000000`	positive integer (checked in `plugin/crew/hooks/scripts/crew_autopilot_backlog.py`)	1.1.21	Input tokens may s assist counte the ru transc be rea Anyth intege 20000 warni
`autopilot.maxLanes`	repo	`null`	positive integer or null (checked in `plugin/crew/hooks/scripts/crew_wave.py`)	1.1.6	Lanes /crew: runs a resolv pm.max larger to it a positiv as it, warni
`autopilot.reviewPolicy`	repo	`"stop"`	`stop` | `clean-only` | `fix-and-rereview`	1.1.6	What with a wave the fir only t round check rerevi ledger single (T-006 rerevi non-fi

Setting	Layer	Default	Values	Since	Summ
					round lines i phase review else r a warn accep

Tickets

Setting	Layer	Default	Values	Since	Summary
`tickets.baseBranch`	repo	`null`	branch name or null (checked in `plugin/crew/hooks/scripts/scope_base.py`)	1.0.158	The branch ticket branches are cut from; null tries origin/HEAD's target, then origin/main, then main. A value that is not a branch name, or names no commit, makes the scope base could not tell.

Route

Setting	Layer	Default	Values	Since	Summary
`route.enabled`	both	`false`	`false` | `true` (checked in `plugin/crew/hooks/scripts/crew_route.py`)	1.0.42	Route plain-text prompts to `/crew:` commands; only the JSON value `true` arms it.

`unattendedCloud`

Setting	Layer	Default	Values	Since
`unattendedCloud.aws.readOnly.profile`	machine-only	`null`	profile name, or null (coerced in `plugin/crew/hooks/scripts/crew_unattended.py`)	1.1.10
`unattendedCloud.aws.readOnly.identity`	machine-only	`null`	ARN prefix ending in `/`, or null (coerced in `plugin/crew/hooks/scripts/crew_unattended.py`)	1.1.10
`unattendedCloud.aws.readOnly.region`	machine-only	`null`	region, or null (coerced in `plugin/crew/hooks/scripts/crew_unattended.py`)	1.1.10
`unattendedCloud.aws.nonProd`	machine-only	`{}`	None (checked in `plugin/crew/hooks/scripts/crew_unattended.py`)	1.1.10

Coming (Not in Code Yet)

Keys from approved tickets that have not landed. Each moves into the table above when its ticket lands, because it is then in the code; a test fails until it does

(`test_no_coming_key_is_in_code`).

T-0030

Setting	Change	Layer	Default	Values	Summary
`coord.ttlMinutes`	new key	set when T-0030 lands	30		Lifetime of a cross-session coordination claim (1-10080).
`coord.channel`	new key	set when T-0030 lands	set when T-0030 lands		Where sessions coordinate.

T-0050

Setting	Change	Layer	Default	Values	Summary
`scope.allowCliApproval`	changes layer	both, stricter wins	false		Becomes settable in the global file, the stricter layer winning, once its reader (`crew_ticket.cli_approval_allow`) reads the global layer in a harness-only follow-up to T-005